An Intelligent Text Encryption System Using Musical Notes



Reman Krishnan, Ryan Thomas, D. S. Akshay, Varsha Sunil Krishna, and R. S. Divya

Abstract Musical cryptograms have always been a matter of fascination for musicians and cryptologists across the world. In such cryptograms, each alphabet could be mapped to musical notes in numerous methods, which makes their detection difficult. The proposed system uses musical passwords to access the files and musical notes to encrypt and decrypt the text file data. When a user accesses a file, a musical keyboard appears and the user is prompted to play the musical password. Users define their own password as a sequence of musical notes of varying length. The musical password further serves as the key during encryption. The output would be a set of musical notes which represents the corresponding text in the file. The encrypted text file is then played as music. The proposed model is a symmetric model, where the same musical password used to encrypt should be used to decrypt data. The system ensures that a given plaintext with a given key generates different encrypted output each time due to usage of random mapping tables.

Keywords Musical cryptography · Encryption · Decryption · Symmetric key cryptography · Musical passwords · Musical notes

R. Krishnan (✉) · R. Thomas · D. S. Akshay · V. S. Krishna · R. S. Divya
Department of Computer Science and Engineering, Mar Baselios College of Engineering and Technology, Trivandrum, Kerala, India
e-mail: reman@gmail.com

R. Thomas
e-mail: rzj.ryan@gmail.com

D. S. Akshay
e-mail: akshaydez01@gmail.com

V. S. Krishna
e-mail: varshanshyam@gmail.com

R. S. Divya
e-mail: divya.rs@mbcet.ac.in

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2021

J. S. Raj et al. (eds.), *Innovative Data Communication Technologies and Application*,
Lecture Notes on Data Engineering and Communications Technologies 59,
https://doi.org/10.1007/978-981-15-9651-3_38

1 Introduction

Cryptography and network security as a branch of computer science have evolved over time, to offer techniques to increase the security of data across networks. Modern cryptographic systems rely on the intractability of certain mathematical problems to ensure security.

Music and its attributes were used in cryptography since olden days in the form of musical cryptograms. A musical cryptogram refers to a musical puzzle, which when compared to normal numeric puzzles, needs more time and skill to be decoded. They have been widely used by many music composers in their music compositions to protect them from being forged.

Music as a tool for cryptography provides some interesting properties, which makes its usage in cryptographic applications a useful one. Some of these interesting features could be listed as below:

- Uniqueness—Unless and otherwise there exists a deliberate attempt, no two tunes usually sound similar completely. This property of uniqueness of music could be used to generate unique musical tunes associated with data for cryptographic purposes.
- Undecipherable—Just by hearing music or by getting the music notes of a tune alone would not be enough to retrieve the data that was encrypted in music. There exist millions of possibilities, which make the system even more secure against a brute-force attack.

In the proposed system, music is used as a tool to encrypt text data and to access files. The system intends to make use of the above-mentioned properties of music to provide better security for text files and its contents.

2 Related Works

Musical cryptography has been a relatively unexplored field within the cryptographic domain. Though musical cryptograms were used by musicians to hide messages within their compositions since olden days, there has not been significant progress. With the advent of modern cryptographic systems like DES, AES, Blowfish, etc., much research has not been done about using musical notes for encryption purposes.

Sams [1] provided insights about the usage of music for cryptographic purposes and how they have been used in historic times. An extensive study about how musical notes were used by various musicians was discussed in his paper. Most researches done on this field emphasized the use of simple substitution techniques that map the English characters to musical notes. Dutta et al. [2] proposed a symmetric cipher that mapped the characters to musical notes with the help of mapping tables generated. The characters were mapped to three octaves of music notes. Dutta et al. [3] proposed a technique to hide messages using musical notes by using 26 alphabets and 10 digits

from 0 to 9, where they were randomly assigned to musical notes. One of the major issues in designing a musical cryptographic model is the difficulty in finding a suitable substitution for characters. Kumar et al. [4] proposed the use of genetic algorithms to generate an optimal sequence of musical notes as encrypted output, which generated more soothing music as output instead of mere random sequences of notes. Yamuna et al. [5] used graph theoretic concepts along with the use of musical notes to encode a binary string. In [5], a CBC mode block cipher was proposed that took musical notes as an initialization vector (IV) and encoded binary strings to musical notes. A graph was constructed with these musical notes and the cryptographic key was derived from the degree of vertices.

Hindustani music and carnatic music, which are two main genres of Indian classical music, have been proposed to be used in cryptography. Rao et al. [6] proposed a cryptographic algorithm that uses carnatic music concepts for encryption. In [6], the characters were mapped to music notes, and the mapping was different for each scale (raga). The raga was proposed to be a part of the key in the proposed system, where the sender and receiver need to agree upon a particular scale(raga) for encryption. Pranav et al. [7] used a Hindustani music scale called Yaman Kalyan for encryption purposes. A semi-natural composition process was proposed in [7] to generate random musical notes based on the raga. Dutta et al. [8] proposed a cryptographic system entirely working on the concept of raga, with no symmetric or asymmetric keys used for encryption and decryption purposes. Dutta et al. [9] proposed a fuzzy logic-based approach to generate a symmetric cipher, which produces music sequences from the candidate notes to encode every character.

Kumar et al. [10] proposed a music cipher using a hybrid of Polybius and Playfair ciphers to encrypt a text and generate a musical output after encrypting. In [10], the key matrix for Playfair cipher was generated using Blum-Blum Shub Generator, a pseudorandom generator and the use of multiple ciphers contributed to the enhanced security. Maity [11] proposed a music cipher using modified Polybius cipher. In [11], Polybius cipher was modified using a 6×6 magic square and musical notes. The 6×6 magic square could accommodate 26 alphabets as well as 10 digits, and the musical notes were used for indexing the rows and columns of the magic square.

Passwords are widely used for user authentication. Usually, passwords are alphanumeric. A wide variety of attacks have been conducted on alphanumeric passwords, which exposed their vulnerability. As an alternative to alphanumeric passwords, graphical passwords were proposed. Suo et al. [12] provide a comprehensive survey regarding graphical passwords. The survey compares various graphical password methods available and their enhanced security has been compared to normal alphanumeric ones. Musical passwords, as well as graphical passwords, have been rarely studied and implemented.

Prakash et al. [13] proposed a voice biometric system, which authenticates a user through the musical notes sung by the user. In [13], the password was taken as the musical pattern sung by the user. Use of musical notes to create a password has been proposed by Kumar [14] in his research paper. The proposed system took musical inputs from users through a virtual keyboard and stored them in a database after

hashing them with the MD-5 hash function. More research needs to be conducted regarding the viability of using musical passwords.

3 Proposed Methodology

The proposed system has been divided into three modules—encryption phase, decryption phase and database management. This section of the paper explains the design and working of each module. The working of each module has been explained and discussed.

3.1 Encryption Phase

Multi-stage encryption has been proposed to increase security—two-phase encryption is proposed. The encryption engine takes the file contents at once as input and generates corresponding encrypted output in the form of musical notes. The encryption module is designed to exhibit properties of a product cipher—by using multiple substitutions and transposition ciphers.

The conversion of plain text, which is made up of English characters, is converted to musical notes with the aid of multiple mapping tables. All characters are converted to discrete numeric values during the first phase of encryption and the numeric value obtained as output from the first phase is converted to musical notes during the second phase of encryption.

Mapping tables are randomly generated for each file. This ensures that different files are encrypted using different mapping tables, which eliminates reuse of mapping tables and eventually increasing security. In-built pseudorandom number generators (PRNG) in programming languages is used for random number generation.

First phase encryption works in two steps. The input contents are reversed, and then each character is mapped to discrete unique numeric values with the help of randomly generated mapping table values. The output of the first phase encryption would be a sequence of numbers (Fig. 1).

Second phase encryption also proceeds in three steps:

- The numeric value sequence obtained as output from the previous stage is XOR ed with the file password user sets.
- The password is converted to its ASCII values and then XOR ed with the numeric sequence.
- A nonlinear function, namely squaring function, is then applied to this XOR output. This alters the length of the numeric sequence. The squaring function takes each two digits of a numeric sequence and replaces them with the square of that two digit number, which would be four digits.

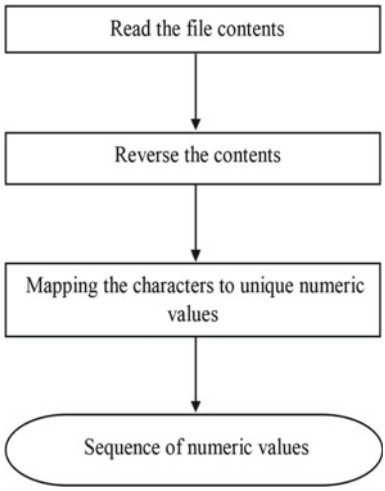


Fig. 1 First phase encryption

This increases the length of cipher text and makes it look different from plain text. The output of the squaring function is then mapped to musical notes with the help of another randomly generated mapping table. The generated music notes are then played (Fig. 2).

Music 21 [15] is a module available in Python language that aids in music generation. This module is imported and used for playing musical notes generated by the encryption engine. Parameters like BPM, duration of musical notes are set according to the user's preference. Once encryption is done, the output file generated is played in media player. This music file is saved at the same location where the encrypted

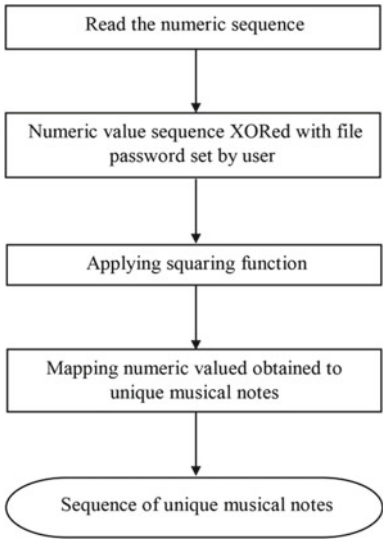


Fig. 2 Second phase encryption

file resides. This enables the user to hear the melody any number of times according to their wish until the file gets decrypted.

The use of XOR operation in second phase encryption changes the numeric sequence in accordance with the ASCII values of the password. XOR operation is done as an intermediate step before the squaring function is applied to the numeric sequence. This step works like a stream cipher—a numeric sequence which acts the plaintext gets XORed with the ASCII values of password, which plays the role of keystream in a standard stream cipher. The insecurity involved in using XOR alone has been eliminated by the application of squaring function, which alters the length of encrypted output before mapping them to musical notes.

3.2 Decryption Phase

The proposed system is a symmetric model, and the decryption is done exactly in reverse order of how encryption was performed. The musical password used to encrypt the file acts as the symmetric key for the proposed system and is used for decryption too. The first phase of decryption proceeds in three steps, as follows (Fig. 3):

- The musical notes are converted back to numeric values, by performing reverse lookup using the same mapping table used for encryption.
- Square root function is then applied, which takes each four digit number in the sequence and replaces it with a square root of that number, which would be a two digit number.

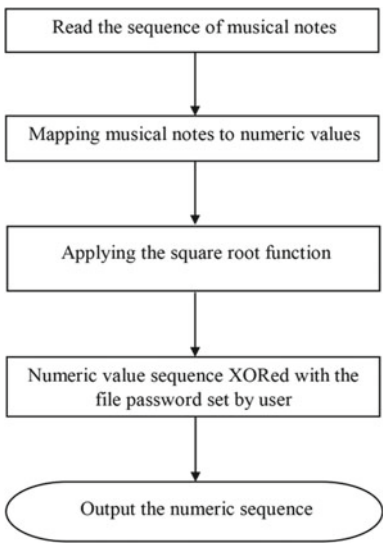


Fig. 3 First phase decryption

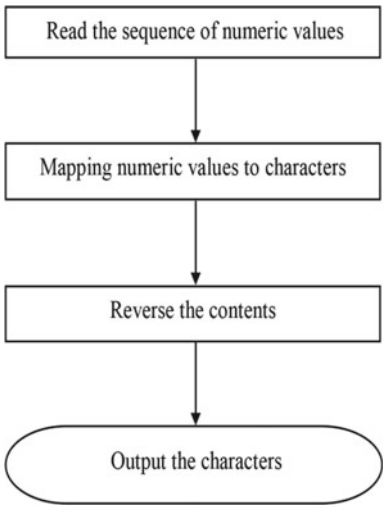


Fig. 4 Second phase decryption

- The numeric sequence obtained is then XOR ed with the file password the user had set. Similar to that in the encryption phase, ASCII values of password characters are used in the decryption phase. This means that the decryption phase cannot decrypt accurately unless the user provides the exact password he/she used for encryption.

The second and final phase of decryption deals with converting the numeric sequence back to the original text. This phase follows two steps (Fig. 4):

- The numeric values are converted back to English characters by performing reverse lookup using the mapping table used for encryption.
- These characters are then reversed to obtain the original file contents back.

3.3 Database Management

To store the credentials associated with each user, a database needs to be maintained. Two database tables are proposed—one table exclusively for storing user's credentials, and others solely for storing information about the files that are presently encrypted.

User's credentials include username, password and their email id. Username and password are required for the user to login to the application. The typed credentials are matched against the database entries for user authorization. A first time user is required to sign up by providing credentials, which would then be stored in its database table. Email id of each user is asked during the sign-up phase, which enables the users to reset their credentials when needed. In such situations, an email would be sent to the user's mail id which instructs steps to reset credentials.

When a user encrypts a file by assigning a musical password, the entry for the corresponding file is created in this database table. Once a file is decrypted successfully, its entry needs to be removed from the database table. This ensures that the table keeps track of presently encrypted files only and also helps in reducing its size.

In certain situations, where the users reset credentials, it needs to be updated in database tables. If a user encrypts a file and then changes their credentials, the password changes. As the file was encrypted using the previous password, the files presently encrypted by the user need to be decrypted, and the file entry should be removed. This ensures consistency of data across database tables.

4 Security Analysis

1. **Brute-force attack on passwords**: Compared to use of alphanumeric passwords, musical passwords are expected to offer a greater degree of security to the proposed system. The proposed system uses a 60 key virtual musical keyboard which takes the musical input from the user or musical password. The password characters are mapped to these 60 musical notes—which cover five octaves. The probability for guessing a music note accurately in the musical password becomes $1/(60!)^n$, where n denotes the length of a password. Guessing becomes even more difficult when more music notes are included in the password, thus generating a highly secure musical password. Since the users can choose any melody of random length, the passwords are extremely difficult to guess accurately.
2. **Database attacks to yield password**: Users create passwords for each file they lock and encrypt. These passwords are stored in a database table. The security of the database tables should not be compromised to a slight degree. To secure passwords, the SHA-2 hash function is being used. The passwords are encrypted using SHA-2 hash, and the digest value is obtained. The security of passwords is further enhanced by converting the message digest obtained to musical notes. The conversion of digest value to musical notes, using the same mapping table which was used in encryption of text contents. This can increase the efforts for cryptanalysis drastically, as the cryptanalyst needs to first decrypt the musical password to obtain hash value.
3. **Frequency analysis of cipher text**: Encryption phase of the proposed system does the job of a substitution cipher. Substitution ciphers have been found vulnerable to frequency analysis attacks. In the proposed system, the vulnerability tends to reduced by ensuring that the same symbols in plain text get encoded to different cipher text. The squaring function applied during the encryption phase ensures that the cipher text and plain text are of different lengths. This enhances security, by eliminating frequency analysis attacks and helps in reducing statistical relationships between plain text and cipher text.
4. **Randomness in encrypted output**: For converting English text to musical notes, two-level mapping is done with the help of two mapping tables. These mapping tables are unique for each file—the probability of two files having the same

mapping table is nearly zero due to randomness involved in its generation. The uniqueness of the mapping table ensures the diversity in the generation of musical notes in cipher text. Mapping tables are concealed from access of users to avoid deletion of mapping tables from the system and to prevent unauthorized access, which otherwise could make the encrypted file vulnerable to cryptanalysis easily, as the cryptanalyst gets access to mapping tables.

Reuse of mapping tables is, as dangerous as a collision in hash functions and attacks could be easily done if an attacker has access to two files with the same mapping tables under any circumstances.

5. **Immunity against standard attacks**: The randomness in encrypted output helps in making the proposed system immune against standard cryptanalytic attacks, which include known plaintext attack, chosen plaintext attack and chosen cipher text attack. The reuse of mapping tables is extremely rare to occur due to pseudo-randomness present, and this means that an attacker needs to attack each text file individually, to get access to each mapping table associated with that particular file.

5 Results and Discussion

The proposed system has been implemented as a Windows-based application, with Python 3.6 used to program encryption and decryption modules. The whole application was built in Microsoft Visual Studio, and the encryption and decryption modules were linked to the application.

Python 3.6 supports random number generation using the *random* module available. Two mapping tables were created, with the aid of in-built random number generator. Ninety-five musical notes were used for encryption purposes, spanning about eight octaves. The characters were mapped to numbers, and these numbers were further mapped to musical notes, randomly. These mappings were used in the first phase as well as in the second phase of encryption. Mapping files were saved and concealed from external access in the system, for the decryption module to use them later. As discussed earlier in the implementation section, Music 21 [15] module has been used for playing the musical notes generated by the file.

To accept musical notes as a password from the user, a musical keyboard was designed as a Windows form, with 60 keys, spanning five octaves. This Windows form was made a part of the whole application. The played musical notes were concatenated as a single string and used as password. This password was sent to encryption and decryption modules. Use of squaring function in the second phase of encryption meant that the encrypted text contains more characters than that in the original file, effectively increasing its size.

Notepad files having size up to 2500 bytes were taken for experimental study and their execution time and output file size were studied. These text files contained English characters as well as digits and special characters. A graph was plotted

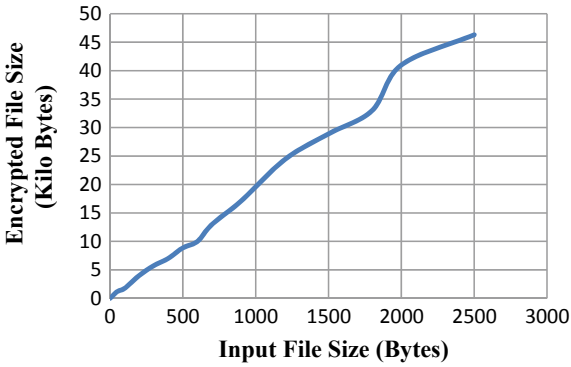


Fig. 5 Encrypted file size versus input file size

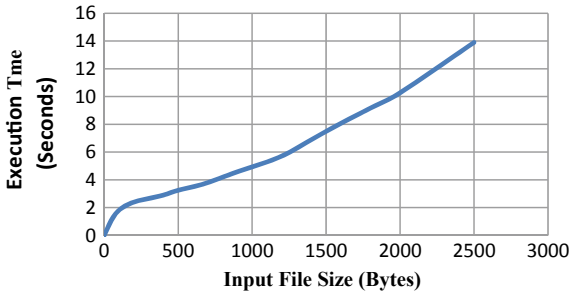


Fig. 6 Execution time versus ınput file size

between input file size (in bytes) versus encrypted file size (in kilobytes). Clearly from the graph (Fig. 5), it can be concluded that the encrypted file size increases largely as the input file size is increased (Fig. 5).

The execution time of the proposed encryption algorithm has been found to increase as the input file size increases. The execution time includes the time taken for encryption as well as the time taken to generate the melody according to the user's parameters. The plot between execution time (seconds) versus input file size (bytes) has been given (Fig. 6). The execution time has been found nearly linear to input file size.

6 Conclusion and Future Scope

The primary goal of encrypting contents of a text file to musical notes and playing the contents as the music was achieved. The difficulty involved in detecting ciphers is enhanced when musical notes are being used for encryption. The accurate guessing of actual characters back from musical notes seems impractical due to randomness involved. Using musical passwords further enhances the security of the proposed system owing to the difficulty involved in exactly guessing the length of musical

password and notes present in password unless a deliberate attempt occurs from the side of the user.

The methodology of using a medium like music, which has been veiled as entertainment and leisure and using it to produce an extensive encryption algorithm which is mathematically secure, highly robust against attacks and theoretically elementary in the realms of cryptography, is atypical.

Music generated by the proposed system lacks an esthetic appeal as the mapping was done randomly. Better algorithms could be incorporated in the proposed system in future which can generate more realistic and esthetic music sequences from the file contents. Further enhancements could be done to the proposed system to make them compatible with the PDF format of files, which are widely used. The proposed system could be extended to support texts in foreign languages other than English.

References

1. Sams E (1979) Musical cryptography. Cryptologia 3(4):193–201
2. Dutta S, Kumar C, Chakraborty S (2013) A symmetric key algorithm for cryptography using music. Int J Eng Technol 5(3):3109–3115
3. Dutta S, Kumar C, Chakraborty S (2010) A novel method for hiding message using musical notes. Int J Comput Appl 1(16):76–79
4. Kumar C, Dutta S, Chakraborty S (2014) Musical cryptography using genetic algorithm. In: 2014 International conference on circuits, power and computing technologies. IEEE, pp 1742–1747
5. Yamuna M, Sankar A, Ravichandran S, Harish V (2013) Encryption of a binary string using music notes and graph theory. Int J Eng Technol 5(3):2920–2925
6. Rao D, Koolagudi S (2019) Music cryptography based on carnatic music. Int J Eng Adv Technol 9(1):5107–5114
7. Pranav P, Chakraborty S, Dutta S (2019) A new cipher system using semi-natural composition in Indian raga. Soft Comput 24(2): 1529–1537 (Springer)
8. Dutta S, Chakraborty S, Mahanti NC (2011) Using Raga as a cryptographic tool. In: International conference on network security and applications 2011. Springer, Berlin, Heidelberg, pp 178–183
9. Dutta S, Kumar C, Chakraborty S (2015) Hidden messages using musical notes: a fuzzy logic approach. Int J Secur Appl 9(1):237–248
10. Kumar C, Dutta S, Chakraborty S (2015) A hybrid polybius-playfair music cipher. Int J Multimedia Ubiqut Eng 10(8):187–198
11. Maity M (2014) A modified version of polybius cipher using magic square and western music notes. Int J Technol Res Eng 1(10):1117–1119
12. Suo X, Zhu Y, Owen GS (2005) Graphical passwords: a survey. In: 21st Annual computer security applications conference (ACSAC'05). IEEE, p 10
13. Prakash R, Kumar S, Kumar C, Mishra KK (2016) Music password based biometric authentication. In: International conference on computing, communication and automation (ICCA2016). IEEE, pp 1016–1019
14. Kumar N (2012) User authentication using musical password. Int J Comput Appl 59(9):1–4
15. Music 21 documentation. https://web.mit.edu/music21/doc/index.html